

WEEK 5: MODEL OPTIMIZATION AND EXPERIMENTATION

Hyperparameter Tuning Strategies for Machine Learning Models

Table of Contents

1.	Introduction & Rationale	1
1.1	Why Model Optimization Matters	1
1.2	What Are Hyperparameters?	1
1.3	The Optimization Process	2
1.4	Why Experimentation is Key	2
2.	Optimization Methods	3
2.1	Grid Search	3
2.2	Random Search	4
2.3	Bayesian Optimization	5
2.4	Comparison Summary	5
2.5	Our Choice: Random Search with Cross-Validation	6
3.	Experimental Design	7
3.1	Parameter Space Definition	7
3.2	Control Variables	7
3.3	Experimental Setup	7
3.4	Sample Code Setup	8
4.	Analysis	9
4.1	Evaluation Metrics	9
4.2	Preventing Overfitting	9
4.3	Expected Improvement	9
5.	Implementation with Python	11
5.1	Complete Optimization Pipeline	11
5.2	Expected Output	16
6.	Expected Outcomes	18
6.1	Expected Improvement	18
6.2	Sample Results Table	18
6.3	Troubleshooting and Challenges	19
7.	Conclusion	20

7.1	Summary	20
7.2	Key Takeaways	20
7.3	Next Steps	20
8.	References	21

1. Introduction & Rationale

1.1 Why Model Optimization Matters

Model optimization is the process of systematically improving a machine learning model's performance by adjusting its hyperparameters. Unlike model parameters (which are learned during training), hyperparameters are set before training and control the learning process itself.

The Impact of Hyperparameter Tuning:

Aspect	Without Tuning	With Tuning
Model Performance	Suboptimal	Near-optimal
Generalization	May overfit/underfit	Balanced
Training Time	May be inefficient	Optimized
Resource Usage	Wasted	Efficient

Real-World Impact:

- Proper tuning can improve model accuracy by **10-30%**
- Reduces training time by **20-50%**
- Prevents overfitting and improves generalization

1.2 What Are Hyperparameters?

Hyperparameter	Description	Example (SVD)
Learning Rate	Controls how much to change weights	lr_all=0.005
Regularization	Prevents overfitting	reg_all=0.1
Number of Factors	Dimensionality of latent space	n_factors=50

Hyperparameter	Description	Example (SVD)
Number of Epochs	Training iterations	n_epochs=20

1.3 The Optimization Process

[Define Parameter Space] → [Select Optimization Method] →

[Run Experiments] → [Evaluate Results] → [Select Best Model]

1.4 Why Experimentation is Key

Reason	Explanation
No Universal Best	Different datasets require different hyperparameters
Interactions	Hyperparameters interact in complex ways
Resource Constraints	Need to balance performance vs. cost
Reproducibility	Systematic experimentation ensures reproducible results

2. Optimization Methods

2.1 Grid Search

Definition: Exhaustively searches through a manually specified subset of the hyperparameter space.

Process:

1. Define a grid of hyperparameter values
2. Train a model for every combination
3. Select the best performing combination

Example (SVD):

```
param_grid = {
```

```
    'n_factors': [20, 50, 100],
```

```
    'reg_all': [0.05, 0.1, 0.2],
```

```
    'lr_all': [0.001, 0.005, 0.01]
```

```
}
```

Total combinations: $3 \times 3 \times 3 = 27$ experiments

Pros and Cons:

Pros	Cons
✓ Simple and exhaustive	✗ Computationally expensive
✓ Guarantees finding best in grid	✗ Scales poorly (curse of dimensionality)
✓ Easy to implement	✗ May miss optimal values between grid points
✓ Reproducible	✗ Wastes resources on poor combinations

When to Use:

- Small parameter spaces (≤ 100 combinations)

- When computation is cheap
- Initial exploration of hyperparameters

2.2 Random Search

Definition: Randomly samples hyperparameter combinations from a defined distribution.

Process:

1. Define distributions for each hyperparameter
2. Randomly sample n combinations
3. Train and evaluate each
4. Select the best

Example (SVD):

```
param_dist = {
    'n_factors': randint(10, 150),
    'reg_all': uniform(0.01, 0.5),
    'lr_all': loguniform(0.0001, 0.1)
}

# Randomly sample 20 combinations
```

Pros and Cons:

Pros	Cons
✓ More efficient than grid search	✗ No guarantee of finding optimal
✓ Can explore larger spaces	✗ Results can be inconsistent
✓ Finds good solutions faster	✗ Requires more runs for confidence
✓ Handles high-dimensional spaces	✗ May miss important regions

When to Use:

- Large parameter spaces
- When computation is expensive
- When you have budget constraints

2.3 Bayesian Optimization

Definition: Uses a probabilistic model (surrogate) to predict which hyperparameter combinations are likely to perform well.

Process:

1. Build a surrogate model (Gaussian Process)
2. Use acquisition function to select next point
3. Evaluate true performance
4. Update surrogate model
5. Repeat

Pros and Cons:

Pros	Cons
✓ Most efficient (fewer evaluations)	✗ Complex to implement
✓ Handles high-dimensional spaces	✗ Requires tuning of surrogate model
✓ Finds near-optimal solutions	✗ Can be computationally intensive
✓ Works well with expensive evaluations	✗ Less interpretable

When to Use:

- Very expensive evaluations
- High-dimensional spaces
- When computational budget is limited

2.4 Comparison Summary

Method	Efficiency	Scalability	Implementation	Best For
Grid Search	Low	Poor	Easy	Small spaces

Method	Efficiency	Scalability	Implementation	Best For
Random Search	Medium	Good	Easy	Large spaces
Bayesian	High	Excellent	Complex	Expensive evaluations

2.5 Our Choice: Random Search with Cross-Validation

For our project, we will use **Random Search with 5-Fold Cross-Validation** because:

1. **Efficiency:** Handles our parameter space (3-5 hyperparameters)
2. **Scalability:** Works well with our dataset size
3. **Robustness:** Cross-validation prevents overfitting
4. **Practicality:** Easy to implement with scikit-learn

3. Experimental Design

3.1 Parameter Space Definition

SVD Model Parameters:

Parameter	Range	Distribution	Why
n_factors	10 - 150	Integer (Uniform)	Number of latent factors
reg_all	0.01 - 0.5	Continuous (Uniform)	Regularization strength
lr_all	0.0001 - 0.1	Continuous (Log Uniform)	Learning rate
n_epochs	10 - 50	Integer (Uniform)	Training iterations

3.2 Control Variables

To ensure fair comparison, we keep these fixed:

Variable	Value	Why
Random Seed	42	Reproducibility
Test Size	20%	Standard split
Cross-Validation	5-Fold	Balance bias-variance
Evaluation Metric	RMSE	Primary metric

3.3 Experimental Setup

Our Experiment Plan:

1. Phase 1: Initial Exploration

- Run 30 random search iterations
- Track RMSE, MAE, Precision@10, Recall@10
- Identify promising regions

2. Phase 2: Focused Search

- Narrow parameter ranges around best values
- Run 20 additional iterations
- Refine results

3. Phase 3: Validation

- Test best configuration with 10-Fold CV
- Compare with baseline
- Document results

3.4 Sample Code Setup

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform, loguniform
# Define parameter distributions
param_dist = {
    'n_factors': randint(10, 150),
    'reg_all': uniform(0.01, 0.5),
    'lr_all': loguniform(0.0001, 0.1),
    'n_epochs': randint(10, 50)
}
# Randomized search with 5-fold CV
random_search = RandomizedSearchCV(
    estimator=SVD(),
    param_distributions=param_dist,
    n_iter=30,
    cv=5,
    scoring='neg_mean_squared_error',
    random_state=42,
    n_jobs=-1
)
random_search.fit(data)
```

4. Analysis

4.1 Evaluation Metrics

Metric	Purpose	Target Value
RMSE	Prediction accuracy	< 0.85
MAE	Average error	< 0.68
Precision@10	Recommendation quality	> 0.35
Recall@10	Coverage	> 0.23

4.2 Preventing Overfitting

Technique	How It Helps
Cross-Validation	Validates on multiple subsets
Regularization	Penalizes complex models
Early Stopping	Stops before overfitting
Validation Curve	Monitors train/test performance

4.3 Expected Improvement

Metric	Baseline	After Optimization	Expected Improvement
RMSE	1.0723	0.86	~20%

Metric	Baseline	After Optimization	Expected Improvement
MAE	N/A	0.67	-
Precision@10	N/A	0.36	-
Recall@10	N/A	0.24	-

5. Implementation with Python

5.1 Complete Optimization Pipeline

```
# =====  
  
# WEEK 5: MODEL OPTIMIZATION PIPELINE  
  
# =====  
  
# Author: Tirumala Karthik  
  
# Date: August 2026  
  
# Description: Hyperparameter optimization for SVD model  
  
# using Randomized Search with Cross-Validation  
  
import pandas as pd  
  
import numpy as np  
  
from surprise import Dataset, Reader, SVD  
  
from surprise.model_selection import cross_validate, train_test_split  
  
from scipy.stats import randint, uniform, loguniform  
  
import warnings  
  
warnings.filterwarnings('ignore')  
  
# =====  
  
# STEP 1: LOAD DATA  
  
# =====  
  
print("Loading data...")  
  
ratings = pd.read_csv('../data/processed/ratings_clean.csv')  
  
reader = Reader(rating_scale=(1, 5))  
  
data = Dataset.load_from_df(ratings[['user_id', 'item_id', 'rating']], reader)  
  
print(f"✅ Data loaded: {ratings.shape[0]} ratings")
```

```

# =====

# STEP 2: BASELINE EVALUATION

# =====

print("\n" + "="*60)

print("BASELINE EVALUATION")

print("="*60)

from surprise import NormalPredictor

baseline = NormalPredictor()

cv_baseline = cross_validate(baseline, data, measures=['rmse'], cv=5, verbose=True)

baseline_rmse = np.mean(cv_baseline['test_rmse'])

print(f"Baseline RMSE: {baseline_rmse:.4f}")

# =====

# STEP 3: RANDOMIZED SEARCH

# =====

print("\n" + "="*60)

print("RANDOMIZED SEARCH (30 iterations, 5-Fold CV)")

print("="*60)

# Define parameter distributions

param_dist = {

    'n_factors': randint(10, 150),

    'reg_all': uniform(0.01, 0.5),

    'lr_all': loguniform(0.0001, 0.1),

    'n_epochs': randint(10, 50)

}

```

```

# Run optimization

results = []

for i in range(30):

    params = {

        'n_factors': int(param_dist['n_factors'].rvs()),

        'reg_all': param_dist['reg_all'].rvs(),

        'lr_all': param_dist['lr_all'].rvs(),

        'n_epochs': int(param_dist['n_epochs'].rvs()),

        'random_state': 42

    }

    cv_results = cross_validate(

        SVD(**params), data, measures=['rmse', 'mae'], cv=5, verbose=False

    )

    results.append({

        **params,

        'mean_rmse': np.mean(cv_results['test_rmse']),

        'std_rmse': np.std(cv_results['test_rmse'])

    })

    print(f"Iteration {i+1}/30: RMSE = {results[-1]['mean_rmse']:.4f}")

# =====

# STEP 4: FIND BEST PARAMETERS

# =====

results_df = pd.DataFrame(results)

best_idx = results_df['mean_rmse'].idxmin()

```

```

best_params = results_df.loc[best_idx].to_dict()

print("\n" + "="*60)

print("BEST PARAMETERS FOUND")

print("="*60)

print(f'n_factors: {best_params['n_factors']}')

print(f'reg_all: {best_params['reg_all']:.4f}')

print(f'lr_all: {best_params['lr_all']:.4f}')

print(f'n_epochs: {best_params['n_epochs']}')

print(f'RMSE: {best_params['mean_rmse']:.4f}')

print(f'RMSE Std: {best_params['std_rmse']:.4f}')

# =====

# STEP 5: FINAL MODEL EVALUATION

# =====

print("\n" + "="*60)

print("FINAL MODEL EVALUATION")

print("="*60)

# Train best model

best_model = SVD(

    n_factors=best_params['n_factors'],

    reg_all=best_params['reg_all'],

    lr_all=best_params['lr_all'],

    n_epochs=best_params['n_epochs'],

    random_state=42

)

```



```

cv_final = cross_validate(best_model, data, measures=['rmse', 'mae'], cv=5, verbose=True)

final_rmse = np.mean(cv_final['test_rmse'])

final_mae = np.mean(cv_final['test_mae'])

# =====

# STEP 6: COMPARE WITH BASELINE

# =====

print("\n" + "="*60)

print("PERFORMANCE COMPARISON")

print("="*60)

print(f'Baseline RMSE: {baseline_rmse:.4f}')

print(f'Optimized SVD RMSE: {final_rmse:.4f}')

print(f'Improvement: {(((baseline_rmse - final_rmse) / baseline_rmse * 100):.2f}%)')

# =====

# STEP 7: SAVE RESULTS

# =====

import pickle

import os

os.makedirs('../outputs/models', exist_ok=True)

# Save best model

with open('../outputs/models/best_svd_model.pkl', 'wb') as f:

    pickle.dump(best_model, f)

# Save results

results_df.to_csv('../outputs/optimization_results.csv', index=False)

print("\n🟢 Best model saved to: outputs/models/best_svd_model.pkl")

```

```
print("✅ Results saved to: outputs/optimization_results.csv")
```

```
print("\n" + "="*60)
```

```
print("✅ WEEK 5 OPTIMIZATION COMPLETE!")
```

```
print("="*60)
```

5.2 Expected Output

Loading data...

✅ Data loaded: 100000 ratings

=====

BASELINE EVALUATION

=====

Baseline RMSE: 1.0723

=====

RANDOMIZED SEARCH (30 iterations, 5-Fold CV)

=====

Iteration 1/30: RMSE = 0.8856

Iteration 2/30: RMSE = 0.8732

Iteration 3/30: RMSE = 0.8724

...

Iteration 30/30: RMSE = 0.8612

=====

BEST PARAMETERS FOUND

=====

n_factors: 80

reg_all: 0.0800

lr_all: 0.0080

n_epochs: 35

RMSE: 0.8612

RMSE Std: 0.0045

FINAL MODEL EVALUATION

RMSE: 0.8625 (+/- 0.0042)

MAE: 0.6712 (+/- 0.0038)

PERFORMANCE COMPARISON

Baseline RMSE: 1.0723

Optimized SVD RMSE: 0.8625

Improvement: 19.57%

✓ Best model saved to: outputs/models/best_svd_model.pkl

✓ Results saved to: outputs/optimization_results.csv

✓ WEEK 5 OPTIMIZATION COMPLETE!

6. Expected Outcomes

6.1 Expected Improvement

Metric	Baseline	After Optimization	Expected Improvement
RMSE	1.0723	0.86	~20%
MAE	N/A	0.67	-
Precision@10	N/A	0.36	-
Recall@10	N/A	0.24	-

6.2 Sample Results Table

Experiment	n_factors	reg_all	lr_all	RMSE
1	20	0.10	0.005	0.8856
2	50	0.10	0.005	0.8732
3	100	0.10	0.005	0.8724
4	50	0.05	0.005	0.8751
5	50	0.20	0.005	0.8748
Best	80	0.08	0.008	0.8612

6.3 Troubleshooting and Challenges

Challenge	Solution
Long Training Time	Reduce number of CV folds, use fewer iterations
Overfitting	Increase regularization, use more data
Inconsistent Results	Set random seed, increase iterations
Memory Issues	Reduce dataset size, use batch processing

7. Conclusion

7.1 Summary

This report has presented a comprehensive framework for model optimization:

1. **Optimization Methods:** Grid Search, Random Search, Bayesian Optimization
2. **Experimental Design:** Parameter space, control variables, iterative process
3. **Analysis:** Metrics, overfitting prevention, validation curves
4. **Implementation:** Complete Python pipeline with Randomized Search

7.2 Key Takeaways

Takeaway	Implication
Optimization is Essential	Proper tuning improves performance by 15-30%
Random Search is Efficient	Best balance of quality and cost for our problem
Cross-Validation is Crucial	Prevents overfitting and ensures generalization
Experimentation is Key	Systematic approach leads to better results

7.3 Next Steps

Week	Focus	Status
Week 6	MLOps & Deployment	Upcoming

8. References

1. Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(Feb), 281-305.
2. Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. *Advances in Neural Information Processing Systems*, 25, 2951-2959.
3. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(Oct), 2825-2830.
4. Bergstra, J., et al. (2011). Algorithms for hyper-parameter optimization. *Advances in Neural Information Processing Systems*, 24, 2546-2554.